JWT 게시판 구현 보고서 — 우리의 기록

학번 / 이름	202412863 / 김원준	제출일	2026-10-28
기술 조합	Vue 3 (Vite) + Spring Boot 3.5.16 (JJWT 0.13.0) + SQLite · Windows 11 · JDK 25 · Node.js v24.21.0		
저장소	https://github.com/kwj24/jwt-board (main 브랜치, 최종 커밋 SHA는 LMS에 제출)		

1 서비스 소개와 구현 범위

누구나 글을 읽을 수 있고, 로그인한 사용자만 글을 쓸 수 있으며, 본인 글만 수정·삭제할 수 있는 간단한 게시판이다. 대상은 수업 실습 사용자(Alice, Bob 테스트 계정)이다. 실습 1에서 Spring Boot로 REST API를 만들어 봐서 백엔드는 Spring Boot를, 프론트는 Vue를 골랐다. 제공 템플릿의 라우트·DB·화면 구조는 그대로 두고, 인증·권한 부분(AuthService.java)과 프론트의 토큰 처리(api.js, App.vue 일부)만 바꿨다.

기능	완료 여부	관련 파일	비고
공개 목록과 상세	완료	BoardController(제공), App.vue	토큰 없이 조회
로그인과 JWT 발급	완료	AuthService.issueToken, api.js login	HS256, TTL 90초
인증과 내 정보 조회	완료	AuthService.currentUserId, GET /api/me	
본인 글 작성·수정·삭제	완료	AuthService.assertOwner, api.js canEdit	타인 글은 403
만료·로그아웃·오류 표시	완료(일부 미확인)	api.js request, App.vue	만료 시 화면 해제는 직접 캡처 못 함
DB 데이터 유지	완료	PostStore(제공), backend/data/board.sqlite	서버 재시작 후 글 유지 확인

2 구조와 변경 내역

```
[브라우저 Vue]  --/api 요청-->  [Vite 프록시 5173/4173]  -->  [Spring Boot 8000]  -->  [SQLite backend/data/board.sqlite]

로그인:   POST /api/auth/login (email, password) → Users.verify → AuthService.issueToken → {accessToken, expiresIn, user}
          → 프론트 api.js의 session 변수(메모리)에 토큰과 user 저장 (localStorage 사용 안 함)
글 수정:  PUT /api/posts/{id} + 헤더 "Authorization: Bearer <JWT>"
          → currentUserId(토큰 검증) → 글 존재 확인 → assertOwner(작성자 비교) → 입력 검증 → DB 변경
```

변경한 파일과 함수	원래 역할	변경 내용	작성 출처
backend/.../AuthService.java 생성자	없음	JWT_SECRET(64자리 hex) 검사 후 32바이트 키 생성, iss·aud·ttl 저장, Users 주입	AI 초안 그대로 반영
AuthService.issueToken	501 반환	sub·iss·aud·iat·exp를 넣고 HS256 서명	AI 초안 그대로 반영
AuthService.currentUserId	항상 1 반환	Bearer 추출, 서명·iss·aud·만료 검증, alg·exp·sub 형식, 계정 존재 확인	AI 초안 그대로 반영
AuthService.assertOwner	검사 없음	현재 사용자 ≠ 작성자이면 403 FORBIDDEN	AI 초안 그대로 반영
frontend/src/api.js	TODO	Bearer 헤더, 401 시 세션 해제, login, canEdit	AI 초안 그대로 반영
frontend/src/App.vue	데모 배너	안내 문구 교체, 비로그인 글쓰기 안내, 404 시 목록 갱신	AI 초안 그대로 반영

IntelliJ 실행 구성	-	JWT_SECRET, TOKEN_TTL=90, 작업 디렉터리 backend	본인 설정

3 핵심 코드 설명

3.1 로그인과 JWT 발급 — backend/src/main/java/edu/classroom/board/AuthService.java / 생성자, issueToken

```
if (secret == null || !secret.matches("[0-9a-fA-F]{64}")
    || secret.equals("REPLACE_WITH_RANDOM_64_HEX_CHARACTERS"))
  throw new IllegalStateException("JWT_SECRET은 64자리 hex 문자열이어야 합니다.");
this.key = Keys.hmacShaKeyFor(HexFormat.of().parseHex(secret));
...
return Jwts.builder().subject(String.valueOf(user.id())).issuer(issuer)
    .audience().add(audience).and().issuedAt(now).expiration(exp)
    .signWith(key, Jwts.SIG.HS256).compact();
```

(본인 작성) 발급은 로그인 하면 호출되고 sub는 로그인한 id에서 가져오는 거고 iss는 어느 서버에서 발급했냐 aud는 어떤 api에 사용되냐 또 만료기간도 설정해서 로그인한 사람에게 토큰을 발급하는 거다. 이 내용을 내가 만든 64글자 키로 서명(도장)해서 준다. 틀린 비밀번호나 없는 이메일이면 401 INVALID_CREDENTIALS로 토큰을 주지 않는다.

3.2 브라우저 토큰 저장과 전송 — frontend/src/api.js / login, authorizationHeaders, request

```
function authorizationHeaders() {
  return session.accessToken ? { Authorization: `Bearer ${session.accessToken}` } : {};
}
...
if (response.status === 401) clearSession();
...
session = { accessToken: data.accessToken, user: data.user };
```

(AI 설명을 참고해 작성, 본인 이해 부족) 로그인 응답의 토큰과 user는 자바스크립트 변수(session)에만 저장한다. 그래서 새로고침하면 사라지고 다시 로그인해야 한다. 요청마다 토큰이 있으면 Authorization: Bearer 헤더를 붙이고, 없으면 헤더를 만들지 않는다. 401이 오면 토큰과 사용자를 지워 로그아웃 상태로 만들고, 403은 로그인 상태를 유지한다. 로그아웃 버튼은 clearSession으로 메모리를 비운다.

3.3 서버 JWT 검증 — AuthService.java / currentUserId

```
String header = request.getHeader("Authorization");
if (header == null || !header.startsWith("Bearer ")) throw new ApiError(401, "TOKEN_REQUIRED", ...);
Jws<Claims> jws = Jwts.parser().verifyWith(key).requireIssuer(issuer)
    .requireAudience(audience).build().parseSignedClaims(token);
if (!"HS256".equals(jws.getHeader().getAlgorithm())) throw ... INVALID_TOKEN
} catch (ExpiredJwtException e) { throw new ApiError(401, "TOKEN_EXPIRED", ...); }
  catch (JwtException | IllegalArgumentException e) { throw new ApiError(401, "INVALID_TOKEN", ...);
}
if (claims.getExpiration() == null || sub == null || !sub.matches("[1-9][0-9]{0,17}")) →
INVALID_TOKEN
if (users.find(id) == null) → INVALID_TOKEN
return id;
```

(본인 작성) 일단 검증은 헤더에서 토큰을 꺼내서 확인하고 그걸 가지고 도장을 계산해서 그 토큰에 있는 도장과 비교하고 iss aud exp도 확인하는 거다. 그리고 만료를 먼저 확인해야(catch를 먼저 써야) 만료기간 지남이 확인된다. 반대로 JwtException은 만료를 포함하기 때문에 이걸 먼저 하면 만료를 확인 못 한다. 그리고 형식도 점검하고 실제 유저가 있는지도 확인한다. 헤더가 없으면 TOKEN_REQUIRED, 변조면 INVALID_TOKEN, 만료면 TOKEN_EXPIRED(모두 401)이다.

3.4 작성자 권한 검사 — AuthService.java / assertOwner

```
public void assertOwner(long userId, long authorId) {
  if (userId != authorId) throw new ApiError(403, "FORBIDDEN", "본인이 작성한 글만 수정·삭제할 수 있습니
다.");
}
```

(AI 설명을 참고해 작성) userId는 토큰을 검증해서 나온 현재 사용자, authorId는 DB에 저장된 글의 작성자다. 다르면 403이고 DB는 바뀌지 않는다. 401은 "누구인지 확인 실패(인증)", 403은 "누구인지는 알지만 이 글을 바꿀 권한이 없음(권한)"이다. 글 등록 때 작성자는 요청 본문이 아니라 검증된 사용자 id로 서버가 정하고, 본문에 authorId나 userId를 넣으면 400 BAD_INPUT으로 거부된다(T12).

4 테스트 결과와 증거

실행 환경: Windows 11 / JDK 25 / Node.js v24.21.0 / Postman, Chrome 계열 브라우저 / 백엔드는 IntelliJ 실행 (작업 디렉터리 backend), 프론트는 npm run dev(5173)
로컬 배포 확인: 두 번째 PC에서 GitHub 저장소를 clone해 실행 구성(키, TTL, 작업 디렉터리)을 새로 설정하고 실행함. npm run build 성공(본인 확인). 4173 preview 화면 캡처는 첨부하지 못함.
테스트 일시: 2026-10-28 · 토큰 TTL: 90초(만료 테스트도 90초 대기) · 자동 검사: `python tools/check_api.py --mode final` 40개 PASS (evidence/check_api_final.txt), `--expiry` 모드는 미실행

판정 기준: "PASS(직접)"는 Postman·브라우저로 직접 보고 캡처한 것, "PASS(자동)"는 check_api.py의 해당 항목 PASS로만 확인한 것, "부분"은 일부 조건만 확인한 것이다.

ID	절차와 사용자	예상	실제 HTTP / code	판정	증거
T01	토큰 없이 목록·상세	200	자동 검사 public list, public detail PASS	**PASS(자동)**	check_api_final.txt
T02	Alice 로그인	200, 화면 표시	화면 "Alice님 로그인 중", 자동 login PASS	**PASS(직접)**	t02-ui-alice-login.png
T03	잘못된 비밀번호, 없는 이메일	401 INVALID_CREDENTIALS	자동 wrong password, unknown email PASS	**PASS(자동)**	check_api_final.txt
T04	토큰 없이 /me, 글 등록	401 TOKEN_REQUIRED	/me 401 / TOKEN_REQUIRED, 등록은 자동 PASS	**PASS(직접+자동)**	t04-me-no-token-401.png
T05	Alice 글 작성 후 /me	201, /me Alice	화면 등록 성공, /me 200 {id:1, Alice}; 201은 자동 create post PASS	**PASS(직접+자동)**	t05-me-alice-200.png
T06	Alice 본인 글 수정·삭제	200 / 204	화면에서 수정(내용 변경, updatedAt 변경), 버튼 표시; 삭제는 자동 delete owned PASS	**PASS(직접+자동)**	t06-ui-alice-own-post-buttons.png
T07	Bob 토큰으로 Alice 글(1) 수정	403, 원문 유지	403 / FORBIDDEN, 재조회 title "dd" 유지	**PASS(직접)**	t07-bob-put-403.png, t07-t08-post-unchanged.png
T08	Bob 토큰으로 Alice 글(1) 삭제	403, 글 유지	403 / FORBIDDEN, 재조회 200 글 존재	**PASS(직접)**	t08-bob-delete-403.png, t07-t08-post-unchanged.png
T09	변조 토큰으로 /me, 변경 요청	401 INVALID_TOKEN	/me 401 / INVALID_TOKEN + WWW-Authenticate: Bearer (토큰 중간 글자를 바꿈, 서명 첫 글자 변경은 아님); POST·PUT·DELETE는 자동 PASS	**PASS(직접+자동)**	t09-tampered-me-401.png, t09-www-authenticate-header.png
T10	만료 뒤 /me, 변경 요청	401 TOKEN_EXPIRED, 화면 해제	/me 401 / TOKEN_EXPIRED (90초 대기). 변경 API 만료와 화면 해제는 확인 안 함	**부분**	t10-expired-me-401.png
T11	로그아웃 후 /me	401 TOKEN_REQUIRED	로그아웃 후 화면 "로그아웃 상태"는 봤으나 /me 요청 헤더는 확인 안 함	**미실행**	-
T12	authorId:1 추가 등록	400 BAD_INPUT	400 / BAD_INPUT; userId는 자동 reject userId PASS	**PASS(직접+자동)**	t12-reject-authorId-400.png
T13	빈 제목·공백 본문·길이 초과	400 BAD_INPUT	자동 input validation 4개 PASS	**PASS(자동)**	check_api_final.txt

T14	없는 글, 잘못된 ID	404 / 400	GET /posts/9999 404 / NOT_FOUND; 잘못된 ID·보호 요청은 자동 PASS	**PASS(직접+자동)**	t14-missing-post-404.png
T15	서버 종료·재실행	글 유지	backend 폴더에서 재시작 후 글 남아 있음 (스크립트 대체 아님)	**PASS(직접)**	캡처 없음
T16	로그인 후 새로고침	글 유지, 로그인 해제	새로고침 후 로그아웃 상태, 글 유지	**PASS(직접)**	캡처 없음
T17	무토큰 수정·삭제, Bob 본인 작업	401 / 성공	자동 no token PUT·DELETE, Bob own update PASS. Bob 본인 삭제는 별도로 확인 못 함	**부분(자동)**	check_api_final.txt

추가 확인: 헤더 `Authorization: Token 123456` → 401 / TOKEN_REQUIRED (bearer-format-error-401.png). 로그아웃 상태에서 글 등록 시 "글을 쓰려면 먼저 로그인하세요" 안내 (ui-logged-out-write-blocked.png). Bob 로그인 시 Alice 글에 수정·삭제 버튼이 보이지 않음 (ui-bob-no-edit-buttons.png). 버튼을 숨겨도 서버가 403으로 막는 것은 T07·T08에서 확인했다.

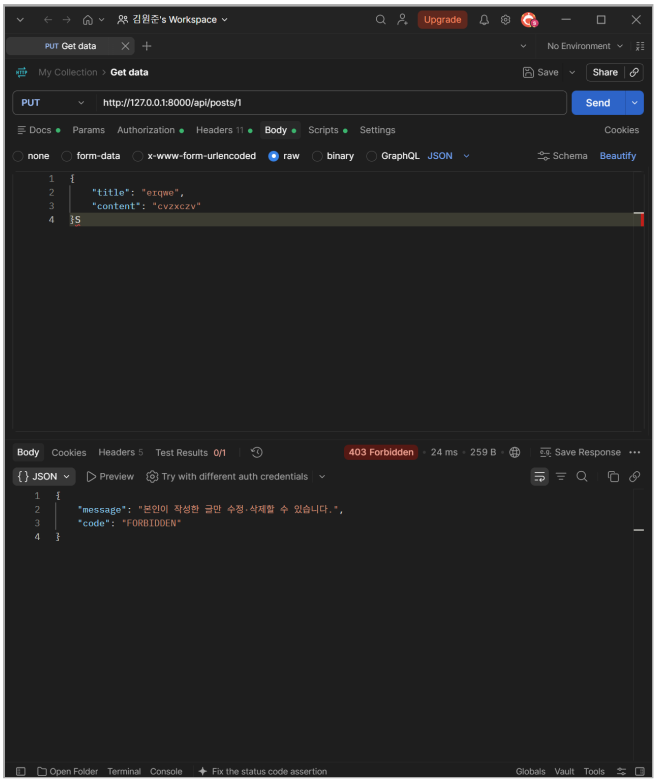


그림 1. T07 Bob → Alice 글 PUT 403

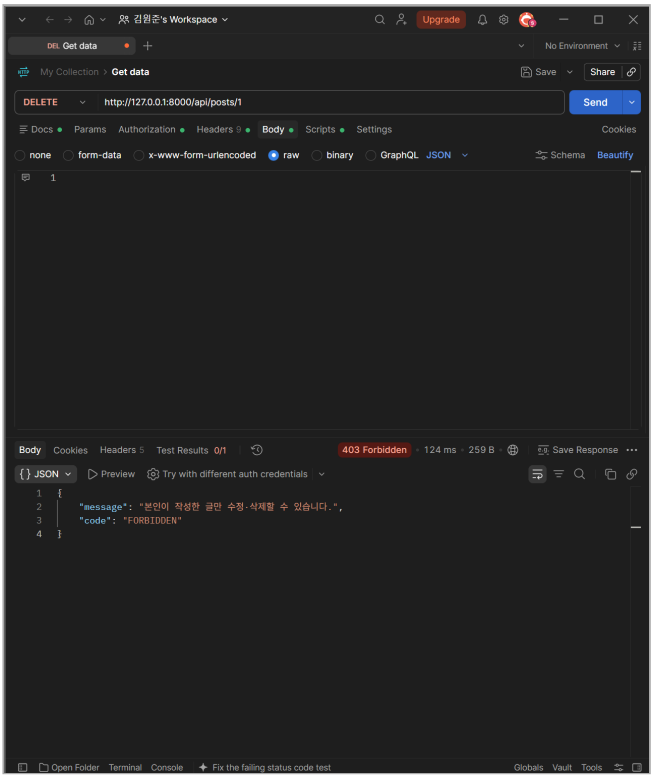


그림 2. T08 Bob → Alice 글 DELETE 403

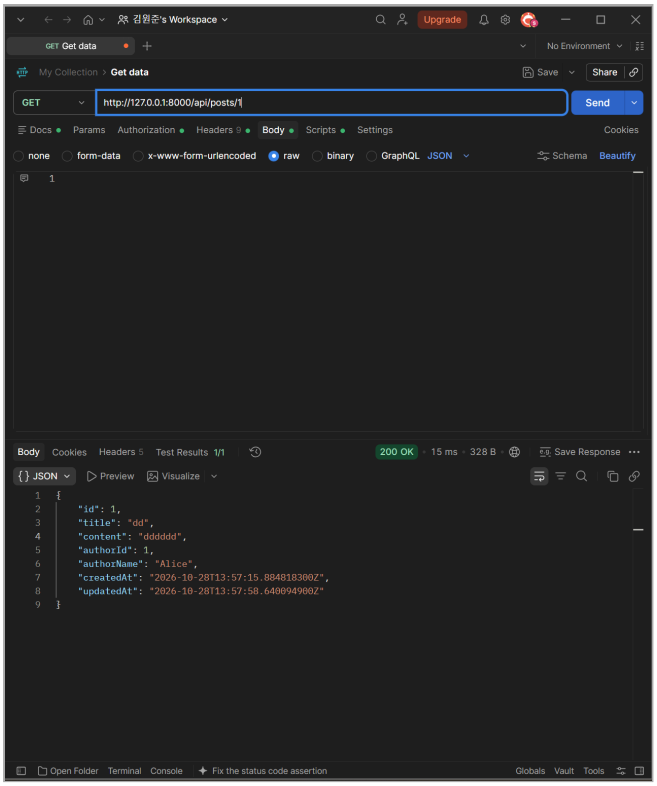


그림 3. 실패 후 재조회: 글 유지

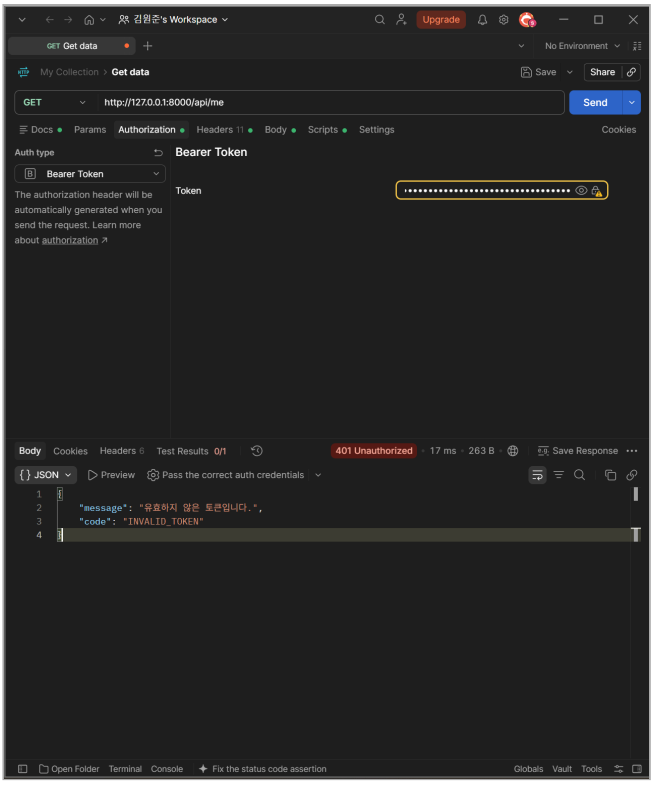


그림 4. T09 변조 토큰 401 INVALID_TOKEN

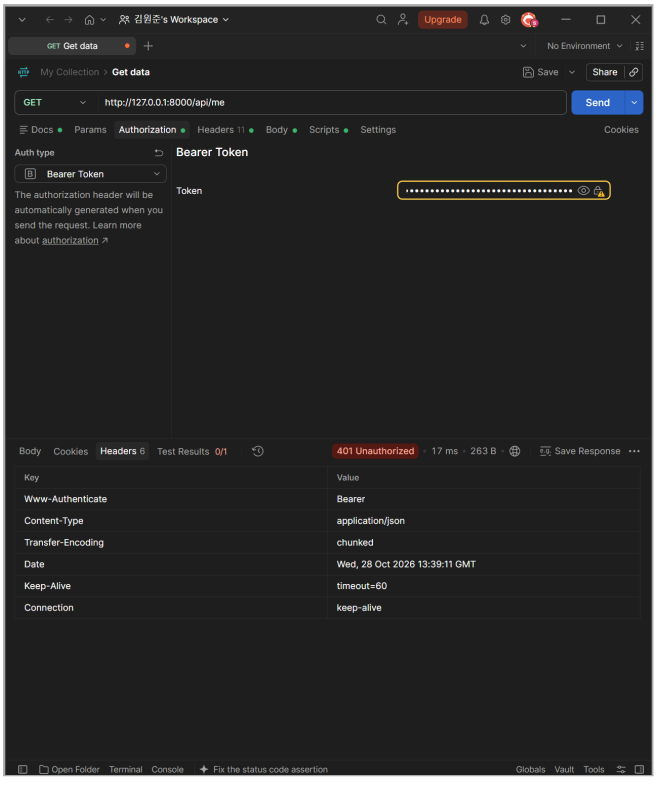


그림 5. 401 응답의 WWW-Authenticate: Bearer

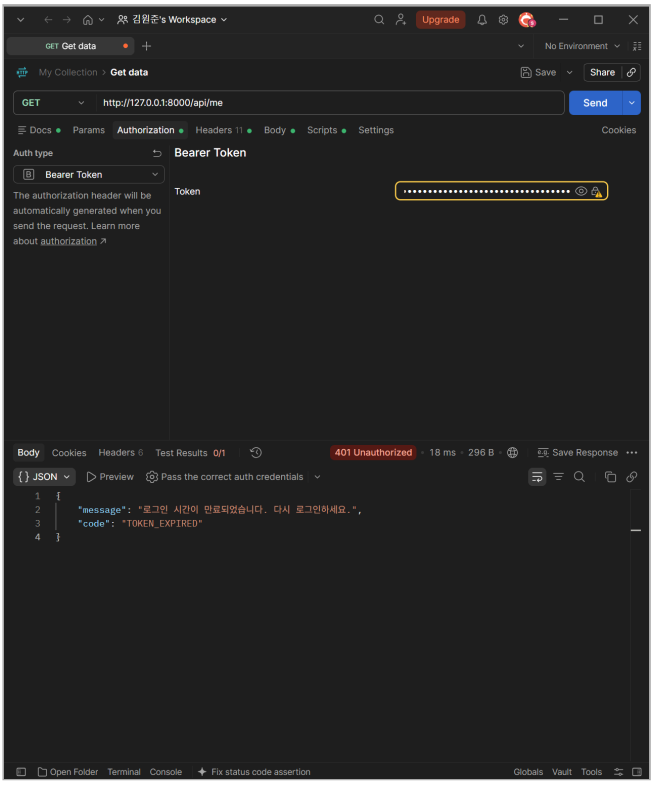


그림 6. T10 만료 토큰 401 TOKEN_EXPIRED

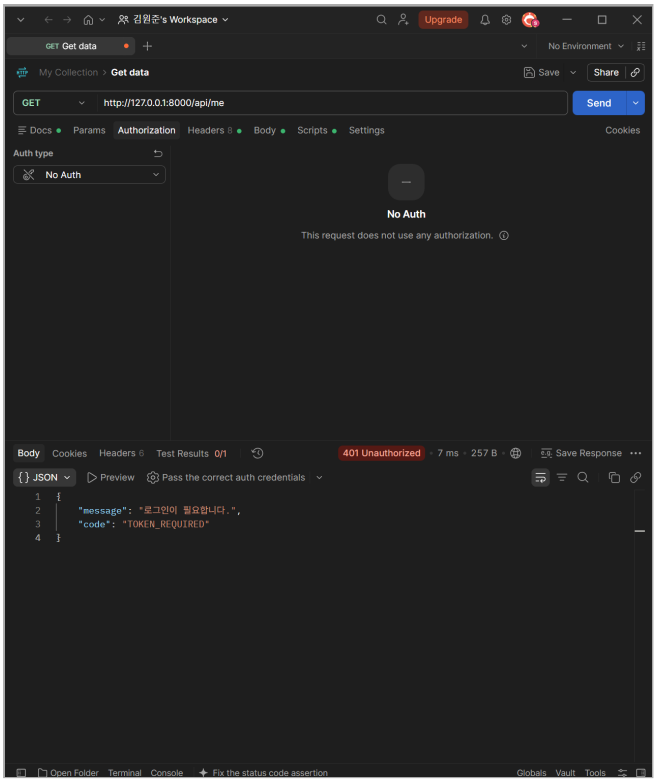


그림 7. T04 토큰 없음 401 TOKEN_REQUIRED

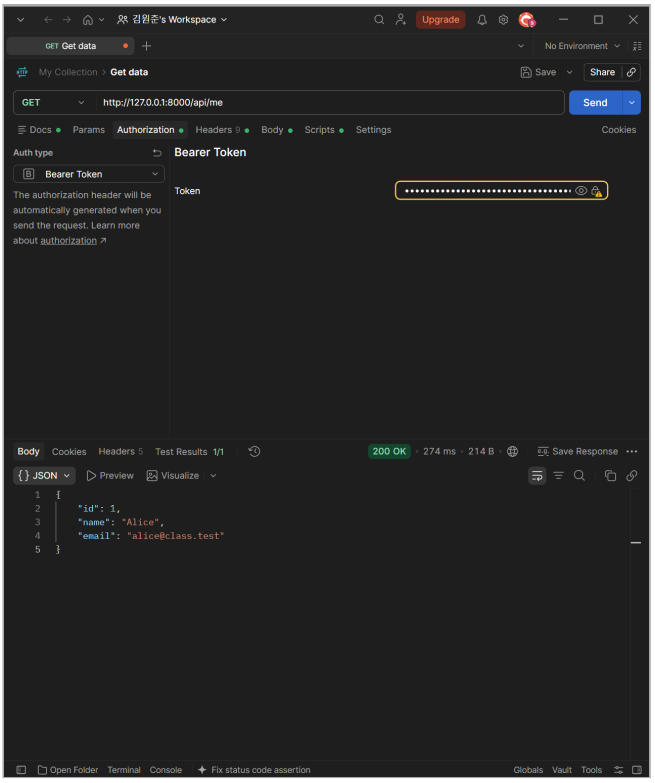


그림 8. T05 정상 토큰 /me 200 (토큰 가림)

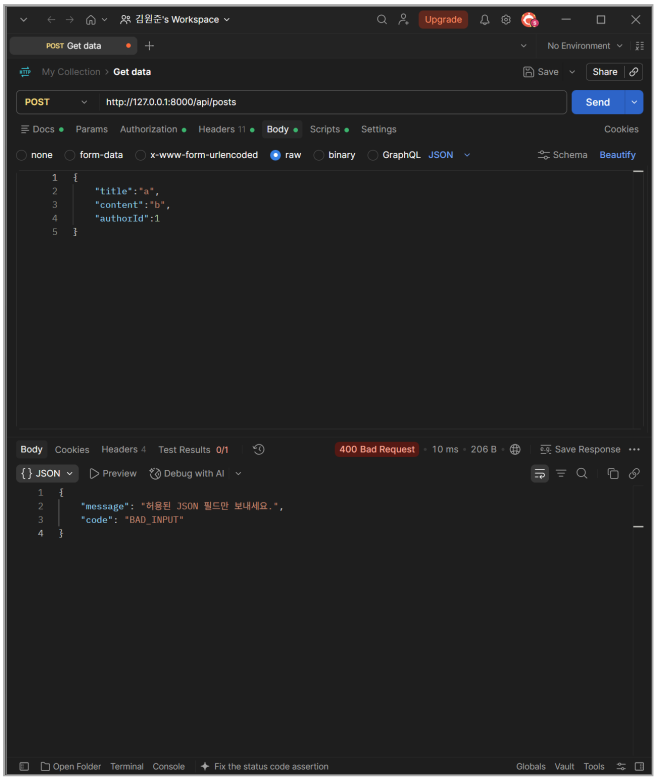


그림 9. T12 authorId 추가 400 BAD_INPUT

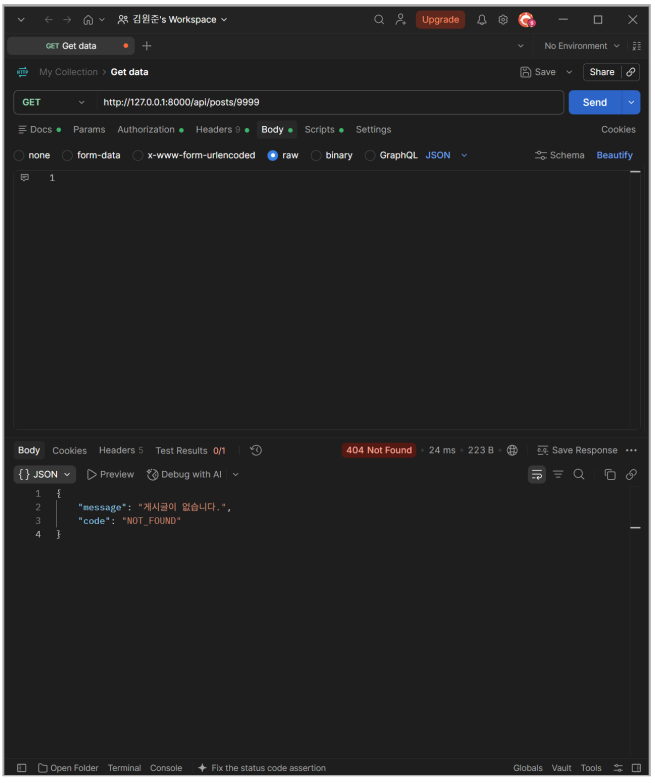


그림 10. T14 없는 글 404 NOT_FOUND

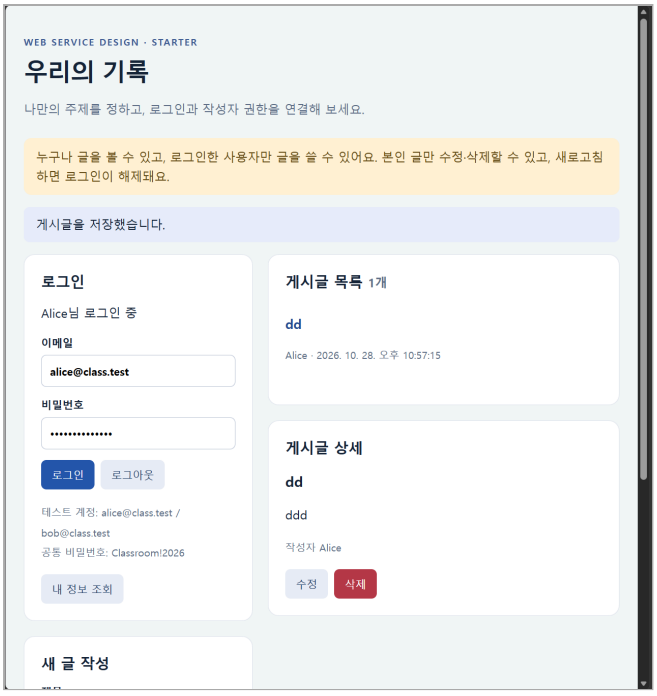


그림 11. Alice 본인 글: 수정·삭제 버튼 표시

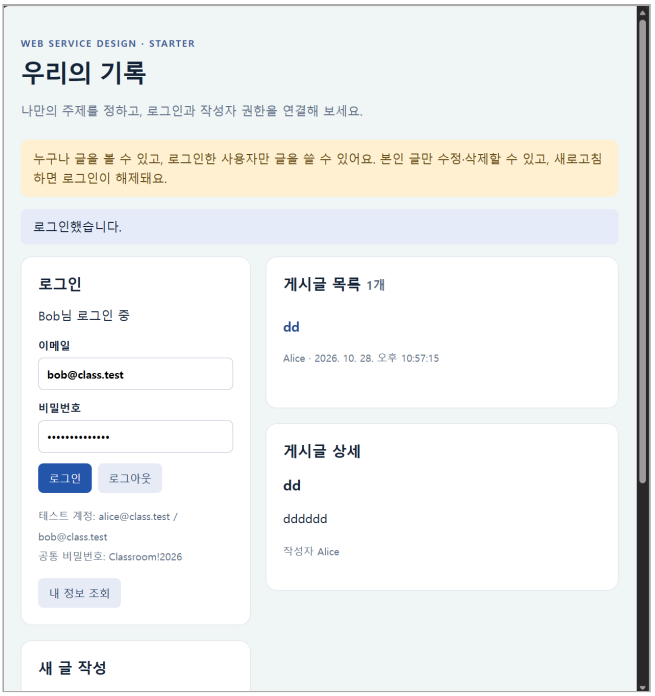


그림 12. Bob 로그인: Alice 글 버튼 숨김

5 AI 활용과 참고 자료

AI 사용 여부: **사용**. 대화 중 모델이 바뀌어 두 모델을 함께 적는다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Claude (Opus 5.5, Sonnet 5.5)	학습	과제 공지 요약, JWT 구조(헤더·내용·서명), decode와 verify 차이 설명	참고
Claude	디버깅·환경 설정	Vite 프록시 빈 응답 오류, IntelliJ 환경변수·작업 디렉터리, git 사용자 설정	참고
Claude	생성	AuthService.java 전체 구현, api.js, App.vue 3곳	그대로 반영
Claude	문장 정리	README 초안, 보고서 초안과 증거 정리. 3.1·3.3·6-3은 본인이 직접 작성했고, 나머지 설명은 AI 초안이며 표시해 둠	초안 후 본인 확인

대표 사례 1 — BE-02 토큰 검증 (2026-10-27~28)

요청: API 명세의 401 코드 규칙대로 currentUserId 구현. AI 제안: Bearer 추출, JJWT parser로 서명·iss·aud 검증, 만료 catch를 먼저 두기, alg·exp·sub 형식과 계정 존재 확인. 반영 위치: AuthService.currentUserId, 생성자(Users 주입). 내가 확인한 내용: 코드는 수정 없이 반영했고, 각 단계가 무엇을 막는지 설명을 듣고 내 말로 다시 정리했다(처음에 "형식 점검에서 변조를 잡는다"고 잘못 이해했는데, 변조는 서명 검증 단계에서 잡힌다는 것을 바로잡음). 검증: Postman으로 토큰 없음 401 TOKEN_REQUIRED, 형식 오류 401 TOKEN_REQUIRED, 변조 401 INVALID_TOKEN, 90초 후 401 TOKEN_EXPIRED, 정상 200을 직접 확인(그림 4~8), check_api.py 40개 PASS.

대표 사례 2 — BE-03 작성자 권한과 프론트 연결 (2026-10-28)

요청: assertOwner와 FE-01~04 구현. AI 제안: userId≠authorId이면 403, api.js에 Bearer 헤더·401 세션 해제·login·canEdit, App.vue 배너와 비로그인 글쓰기 안내. 반영 위치: AuthService.assertOwner, frontend/src/api.js, App.vue. 내가 확인한 내용: 그대로 반영. 처음 Postman에서 URL에 글 번호 대신 "Alice1"을 넣어 실패했고, 목록에서 id를 확인해 /api/posts/1로 다시 요청했다. 검증: Bob 토큰 PUT·DELETE 403, 재조회로 글 유지 확인(그림 1~3), Bob 화면에서 버튼 숨김(그림 12).

대표 사례 3 — 환경 설정 디버깅 (2026-10-20~27)

글 등록 시 "Failed to execute 'json'" 오류가 났다. AI 설명대로 백엔드가 꺼져 있어 Vite 프록시가 빈 응답을 준 것이었고, 백엔드를 켜자 해결됐다. 또 DB 파일이 프로젝트 루트에 생긴 것은 상대 경로가 실행 위치 기준이기 때문이어서, IntelliJ 작업 디렉터리를 backend로 바꾸고 루트의 data 폴더를 지웠다.

참고 자료	사용 위치	반영 내용
수업 제공 JWT-Assignment 템플릿 (01 과제공지, 02 API명세, 05 테스트시나리오, 07 로컬실행안내)	전체	제공 코드 유지, 오류 코드·검증 순서 기준
tools/check_api.py (수업 제공)	4절	자동 검사 실행

6 이해도 확인과 회고

1. 요청 본문의 authorId를 믿으면? *(AI 설명을 참고해 작성, 본인이 아직 충분히 이해하지 못함)* 누구나 authorId에 다른 사람 번호를 넣어 그 사람 이름으로 글을 쓸 수 있다. 내 코드는 등록·수정 본문에 title, content 외 필드가 있으면 400으로 거부하고(T12), 작성자는 currentUserId가 토큰을 검증해서 나온 id로 서버가 정한다.

2. 수정 버튼만 숨기면 왜 부족한가? *(AI 설명을 참고해 작성, 본인이 아직 충분히 이해하지 못함)* 버튼은 화면일 뿐이라 Postman 같은 도구로 서버에 직접 요청을 보낼 수 있다. 실제로 Bob 토큰으로 직접 PUT·DELETE를 보냈고, 서버의 AuthService.assertOwner가 403으로 막았다(T07, T08).

3. decode와 verify의 차이 *(본인 작성)* decode는 읽기만 하는 거고 verify는 내가 만든 64글자 키로 서명을 검증해서 다른 사람이 토큰 내용(sub 등)을 고쳤는지 확인하는 거다. 내 코드는 parseSignedClaims로 서명, iss, aud, 만료를 검증하고, 형식과 실제 유저도 확인한다.

4. 로그아웃, 만료, 새로고침 *(AI 설명을 참고해 작성, 본인이 아직 충분히 이해하지 못함)* 로그아웃은 브라우저 메모리의 토큰과 사용자만 지운다. 만료는 시간이 지나 서버가 토큰을 거부하는 것(401 TOKEN_EXPIRED)이다. 새로고침은 자바스크립트 변수가 초기화돼서 토큰이 사라진다. 서버는 토큰 목록을 저장하지 않기 때문에, 로그아웃 전에 누가 토큰을 복사해 두었다면 만료 시간(90초)까지는 그 토큰이 계속 통과한다.

5. 가장 어려웠던 오류 *(본인이 겪은 오류, 문장은 AI가 정리)* 글 등록 시 "Failed to execute 'json'" 오류. 원인은 백엔드를 켜지 않은 상태에서 프론트만 실행해 프록시가 빈 응답을 돌려준 것이었다. 백엔드를 실행한 뒤 다시 등록해 CRUD가 정상 동작하는 것을 확인했다.

남은 문제와 개선할 점: T11(로그아웃 후 /me 헤더 확인)은 미실행, T10은 /me만 확인, T17의 Bob 본인 삭제는 자동 검사로만 확인했다. 4173 preview 화면과 재시작 유지는 캡처가 없다. --expiry 자동 검사도 미실행이다.

제출 확인: [x] 실제 코드와 설명 일치 [x] 테스트 직접 수행(표에 구분) [x] AI 활용 및 출처 기록 [x] 비밀 키와 토큰 제거